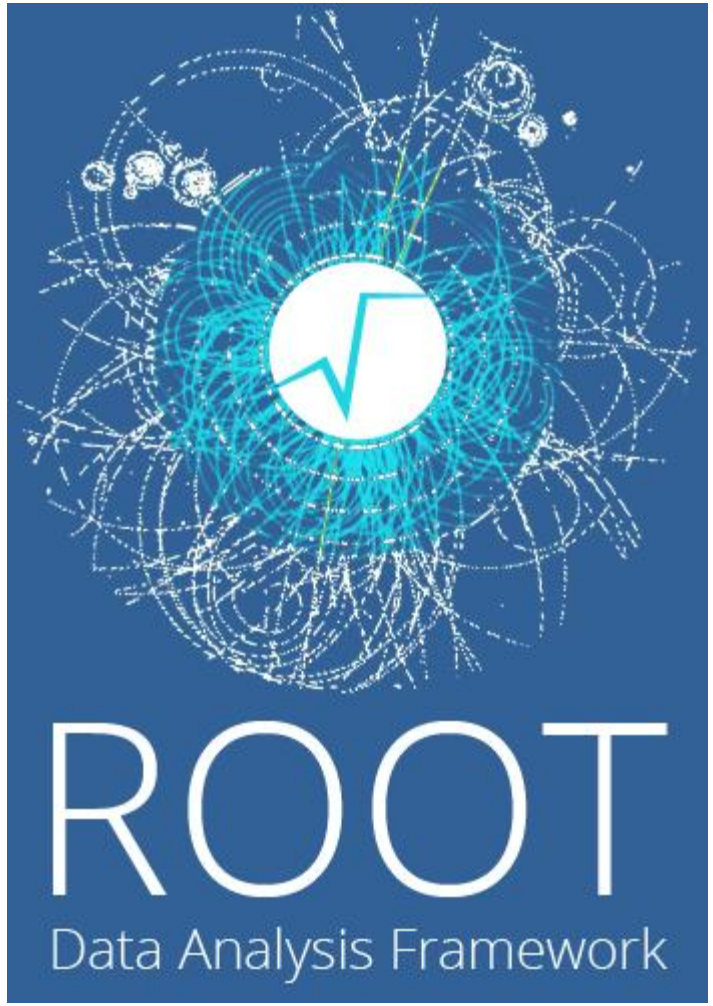
STFC – Nuclear Physics Summer School ROOT workshop

Session 2 - Random numbers and Toy models



Aberdeen University
30/31st July 2026
Simon Gardner (University of Glasgow)
Simon.Gardner@glasgow.ac.uk

Introduction – Random numbers

Hopefully everyone understands the need for random number generators in our field.
Often we are required to generate thousands of random numbers to model single MC events
Repeated random number generation is expensive so it is often a focus of optimization.

ROOT provides an interface to random number generators but the backend algorithm is through standard C++ libraries.
The interface however is powerful when interacting with ROOT objects and modelling physics based distributions.
There are several random number generators available, for details of speed and period see:
ROOT: TRandom Class Reference

To generate a random number in the range 0-1

```
TRandom3 rng2;
double randomnumber = rng2.Rndm();
std::cout << "Random number: " << randomnumber << std::endl;
```

Introduction – Seeds and arrays

Please pull changes to the git repo or re-clone since the previous session:
https://github.com/simonge/STFC-NP_Summer-School-2026_ROOT.git

Run the script RandomGen.C several times, what do you note?

- Now try with a seed argument e.g. root RandomGen.C\(10\)

Seeds

When coding it is important to understand the effects of changes to your code and separate them from effects of random sampling.

```
TRandom3 rng2(seed);
rng2.SetSeed(seed);
```

Without an argument TRandom3 is initialized with seed 65539. Setting the seed to 0 asks it to set the seed based on the time. Advantages to both but understanding can be key to debugging.

Arrays

ROOT provides the function:

```
rng2.RndmArray(N, v.data());
```

Which bulk generates random numbers. In my experience this is 2x faster as demonstrated by the script.

Introduction – Distributions

The ROOT random number generator has methods to automatically transform the random numbers into useful distributions.

Exp(Double_t tau)
Integer(UInt_t imax)
Gaus(Double_t mean, Double_t sigma)
Rndm()
Uniform(Double_t)
Landau(Double_t mean, Double_t sigma)
Poisson(Double_t mean)
Binomial(Int_t ntot, Double_t prob)

You can also sample from a custom function defined as a TF1/2/3

ROOT: TF1 Class Reference

```
TRandom3 rng(123);
TF1 f("f", "gaus", -5, 5);
double x = f.GetRandom(&rng);
```

Or even skipping to filling a histogram randomly

```
//Define a TF1 function with two parameters
auto f1 = new TF1("fa","gaus",-3,3);
f1->SetParameters(0.2,1.3,1);
f1->Draw();

auto h1 = new TH1F("h1","test",100,-3,3);
h1->FillRandom("fa",2000);
```

Note – if you don't provide a random generator ROOT will use an internal one gRandom

Task – Random distributions

- Run myFunc.C and look at the output.

ROOT: TF1 Class Reference

Editing or copying myFunc.C

- Create a three separate functions with parameters of your choice
  - Gaussian
  - BreitWigner
  - Linear
- Fill a single histogram with different number of events from each distribution
- Create a single function which is the sum of the three distributions
  - Ensure your initial parameters estimates are set appropriately
  - Fit your histogram with the new function, compare the fit parameters and magnitudes with your inputs.

Task - Full toy model analysis

Finally, we will try and develop a full analysis of a toy dataset. This is based around polarized photoproduction off a proton where the final goal is to extract the energy-theta dependance on the phi asymmetry.
This is based conceptually my PhD analysis.

- The script ToyGenerator.C will create a dataset containing branches
  - theta – polar angle of some decay products
  - phi – azimuthal angle of some decay products
  - missingMass – mass missing from final state when proton excluded
  - photonEnergy – energy of reaction photon
  - time – coincidence time between photon and reaction

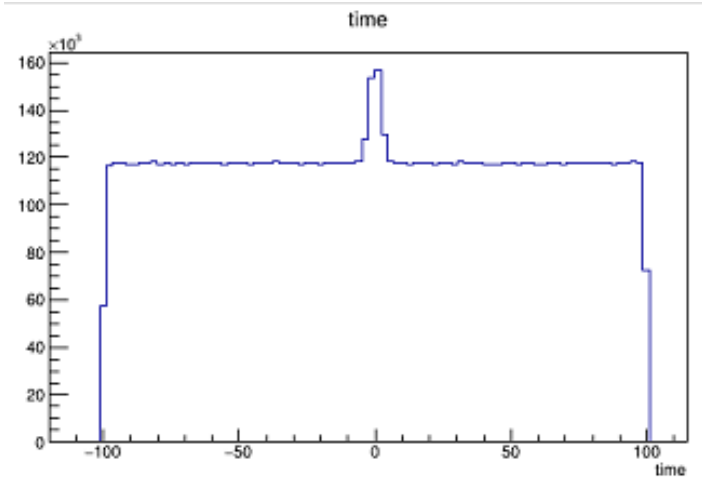
This is still not perfectly physically correct dataset but will suffice.

Each final state comes with uncertainty in the identity of the photon which caused the reaction as several are present in the time window, so we need to carry out a prompt-random statistical background subtraction

To correctly reconstruct the phi asymmetry, you will need to:

- Carry out a weight the events based to timing information
- Divide up the data into theta and energy bins
- Fill histograms with the weighted values of phi
- Carry out a fit of N(1+Acos(2phi)) to determine A for each bin.
- Plot the fit values in a graph with errors.

An AI generated analysis script ToyAnalysis.C is available as a basis but doesn't plot the most useful



Note – The approach to binning and weighting events isn't the most modern approach but is sufficient here.